Natural Language Processing
TP 3 – Réseaux de Neurones Récurrents
Appliquer les RNNs pour la détection de Fake News et pour un Use Case personnalisé

TP noté (Évaluation continue)

Vous devez soumettre vos livrables sur BOOSTCAMP, en respectant la date limite fixée par votre formateur. Soumettez votre propre travail. La triche ne sera pas tolérée et sera pénalisée.
Cette activité peut être réalisée en groupe (jusqu'à 4 membres par groupe).

Réseaux de Neurones Récurrents (RNN)

Les Réseaux de Neurones Récurrents (RNN) sont une classe spéciale de réseaux de neurones conçus pour traiter des données séquentielles. Contrairement aux réseaux de neurones traditionnels feedforward, les RNN peuvent retenir l'information provenant des entrées précédentes grâce à leur état interne (mémoire). Cela les rend particulièrement efficaces pour des tâches où le contexte et l'ordre importent, comme le traitement du langage naturel (NLP), la reconnaissance vocale et la prévision de séries temporelles.

Rappel rapide sur l'analyse de séries temporelles
Une série temporelle est une suite d'observations enregistrées à des points successifs dans le temps (par exemple : prix des actions, température, trafic d'un site web).
L'analyse des séries temporelles permet d'extraire des motifs, tendances et saisonnalités, qui peuvent être utilisés pour la prédiction et la prévision.
Série temporelle univariée : basée sur une seule variable (ex. : ventes journalières).
Série temporelle multivariée : basée sur plusieurs variables (ex. : température, humidité et pression pour prévoir le temps).

Les RNN, et surtout leurs variantes améliorées comme LSTM (Long Short-Term Memory) et GRU (Gated Recurrent Unit), sont largement utilisés dans la prévision de séries temporelles univariées et multivariées.

Objectifs d'apprentissage :

- Comprendre et appliquer le **prétraitement du texte** pour le machine learning.
- Concevoir et entraîner un **RNN (ou LSTM)** sur un cas NLP, comme la classification de texte.
- Expérimenter avec différents **hyperparamètres** et architectures de modèles séquentiels.
- Évaluer le RNN avec des **métriques pertinentes**.

Cas d'usage 3.1 : Prévision du prix des actions Google avec un RNN

Cette activité de laboratoire consiste à prédire le prix des actions de Google à l'aide de réseaux LSTM (Long Short-Term Memory).

Imaginez vivre dans un monde où vous connaissez exactement le prix des actions dans lesquelles vous avez investi, avant même que cela ne se produise. Vous savez exactement quand elles vont monter, exactement quand elles vont baisser. Vous seriez sur la voie pour devenir millionnaire.

Rappel rapide sur la prévision de séries temporelles univariées ou multivariées

Univariée : vous prévoyez une variable future basée uniquement sur ses valeurs passées (ex. : prévision des ventes annuelles basée sur les ventes des années précédentes).

Multivariée : vous prévoyez les valeurs futures d'une variable cible en fonction de ses valeurs passées et des valeurs d'autres variables.

Dans cette activité, nous allons implémenter un modèle RNN de type LSTM pour prévoir le prix d'ouverture des actions de 2017 sur une structure de données avec 60 pas de temps, ce qui signifie : prévoir le prix d'ouverture à l'instant t en se basant sur les 60 valeurs précédentes à [t-60, t-1]. Il s'agit d'une prévision univariée : une seule variable cible (prix d'ouverture) en fonction de ses valeurs historiques.

Ce même cas peut être implémenté en prévision multivariée. Référez-vous aux supports supplémentaires fournis pour comprendre cette implémentation.

Deux jeux de données sont fournis : Google_Stock_Price_Train.csv et Google_Stock_Price_Test.csv.

La solution d'implémentation du réseau LSTM pour prédire le prix des actions de Google est partagée à des fins pédagogiques.

Cas d'usage 3.2 : Détection de Fake News avec RNN (Guidé, non noté)

Cette activité vise à classer les fake news et les vraies news à l'aide d'un réseau neuronal récurrent. Pour simplifier le prétraitement du texte, les fonctions intégrées de TensorFlow seront utilisées au lieu de bibliothèques comme NLTK.

Il s'agit d'une **classification binaire**.

Jeu de données :

- Fake.csv : 23 502 articles de fake news
- True.csv : 21 417 articles de vraies news

Colonnes du dataset :

- Title : titre de l'article
- Text : texte complet de l'article
- Subject : sujet de l'article
- Date : date de publication
-

Crédits à Xingyu Bian / GitHub user: therealcyberlord

Sources :

- https://github.com/therealcyberlord/Fake-News-Detection-Using-RNN?tab=readme-ov-file
- https://www.kaggle.com/code/therealcyberlord/fake-news-detection-using-rnn/notebook

Étapes suivies pour implémenter le classificateur RNN :

1. **Chargement des données**
   - Chargement des deux fichiers CSV : fake news (Fake.csv) et vraies news (True.csv).
2. **Nettoyage et prétraitement**
   - Vérification des valeurs manquantes.
   - Suppression des colonnes non pertinentes (date, subject) pour éviter les biais.
   - Fusion des fake et real news avec étiquettes.
3. **Prétraitement du texte**
   - Suppression des caractères et symboles indésirables via regex.
   - Conversion en minuscules.
   - Tokenisation et padding avec Tokenizer et pad_sequences de TensorFlow.
4. **Division du dataset**
   - Séparation en ensembles d'entraînement et de test avec train_test_split.
5. **Conception du modèle (RNN)**
   - Couche d'**Embedding** pour convertir les mots en vecteurs denses.
   - Couche **SimpleRNN/LSTM** pour capturer les dépendances séquentielles.
   - Couches **Dense** pour la classification.
6. **Entraînement du modèle**
   - Compilation avec optimiseur adam et fonction de perte binary_crossentropy.
   - Entraînement sur le set d'entraînement et validation sur le set de test.
7. **Évaluation**
   - Mesure de l'accuracy, précision, rappel.
   - Construction d'une **matrice de confusion** (en pourcentage).
8. **Résultats**
   - Le modèle atteint une **classification quasi parfaite** entre fake et real news :
     - Accuracy : ~99 %
     - Précision : ~98,8 %
     - Recall : ~99,2 %

Cas d'usage 3.3 : Appliquer un RNN à un cas NLP personnalisé (partie notée)

L'objectif principal est de renforcer votre compréhension des modèles séquentiels et du prétraitement du texte.

Consignes :

1. Sélectionnez un jeu de données textuel (ex. : analyse de sentiment, critiques de films, classification de sujets). (10 points)
2. Prétraitez les données (tokenization, padding, embedding). (25 points)
3. Concevez et implémentez un modèle RNN pour résoudre la tâche. (25 points)
4. Évaluez la performance et discutez des résultats. (20 points)
5. Expérimentez avec des variantes LSTM ou l'optimisation d'hyperparamètres : essayez différentes combinaisons pour améliorer la performance (accuracy, loss, F1-score…) (20 points)

Hyperparamètre	**Fonction**
Nombre d'unités (taille cachée)	Nombre de neurones par couche LSTM
Nombre de couches	Profondeur du réseau (simple vs empilé)
Taux d'apprentissage	Vitesse à laquelle le modèle met à jour les poids lors de l'entraînement
Taille du batch	Nombre d'échantillons par étape d'entraînement
Taux de dropout	Fraction de neurones ignorés aléatoirement pendant l'entraînement
Longueur de séquence / fenêtre d'entrée	Nombre de pas de temps que le LSTM voit à la fois

NB : n'oubliez pas d'ajouter des commentaires internes d'interprétation des résultats et d'explication du code à chaque étape. Ceci est obligatoire et sera noté.

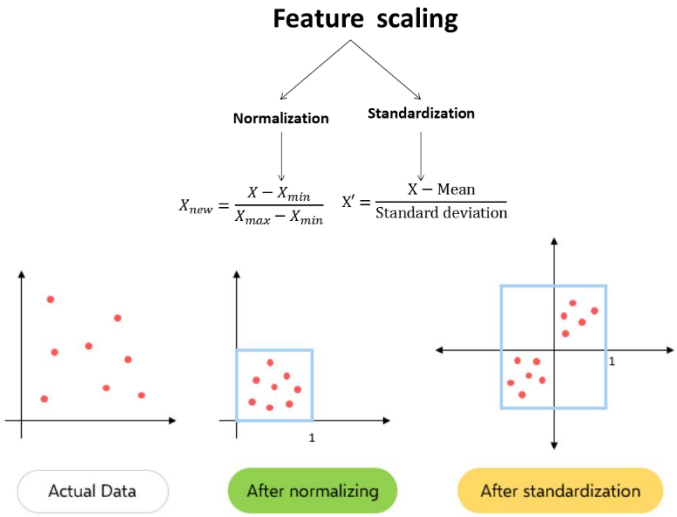
Livrables attendus :

- Le **jeu de données**
- Un **Notebook Python** comprenant :
  - La description de votre cas d'usage
  - Le code source
  - Les sorties (tableaux, graphiques, résultats imprimés)
  - Des commentaires internes expliquant et interprétant les résultats

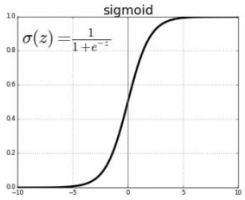
Bonne Chance !

Pour vous aider, voici quelques recommandations pour implémenter un RNN/LSTM. Note : ce ne sont pas des règles absolues, mais des **directives générales** applicables dans la plupart des cas.

<u>Mettez vos données à l'échelle en utilisant la fonction de scaling appropriée :</u>



Il est recommandé d'utiliser la **normalisation** lors de l'implémentation d'un RNN (car la fonction d'activation **Sigmoid** est l'une des principales fonctions utilisées dans les RNN).



<u>Attention : Lors de l'utilisation de RNN / LSTM pour des tâches NLP, on ne met pas généralement à l'échelle les données textuelles de la même manière que l'on met à l'échelle des variables numériques pour la régression ou la prévision de séries temporelles. À la place, on utilise des représentations par embeddings.</u>

La mise à l'échelle est importante lorsque votre RNN/LSTM travaille sur des **données numériques de séries temporelles** (par exemple : prix d'actions, données météorologiques, signaux de capteurs IoT).

- Dans ce cas, on utilise généralement un **Min-Max Scaling** ou une **Standardisation** pour que les valeurs restent dans une plage raisonnable (ex. : [0,1] ou moyenne=0, écart-type=1).
- Cela **améliore la convergence** car les fonctions d'activation comme sigmoid ou tanh sont sensibles aux grandes valeurs d'entrée.
- **Exemple (prétraitement de séries temporelles) :**

  from sklearn.preprocessing import MinMaxScaler

  scaler = MinMaxScaler(feature_range=(0,1))

```
scaled_data = scaler.fit_transform(time_series_data.reshape(-1,1))
```

Pour le NLP (données tectuelles):

- Le texte brut est **tokenisé** (converti en séquences d'entiers).
- Ces entiers sont ensuite passés dans une **couche Embedding** (embeddings appris ou préentraînés comme GloVe, Word2Vec ou BERT).
- Comme les embeddings produisent déjà des vecteurs avec des **plages cohérentes** (souvent de petites valeurs, par exemple entre -1 et 1 ou proches de 0), **aucune mise à l'échelle supplémentaire n'est nécessaire**.

Example:

```
model = tf.keras.Sequential([
  tf.keras.layers.Embedding(input_dim=10000, output_dim=128, input_length=200),
  tf.keras.layers.LSTM(64),
  tf.keras.layers.Dense(1, activation='sigmoid')
])
```

Here, the **Embedding layer** automatically maps words into dense vectors suitable for the LSTM.

<u>La construction et l'entraînement d'un LSTM / RNN incluent les étapes suivantes :</u>

1. Commencez à construire votre réseau neuronal récurrent LSTM.

2. Importez les bibliothèques nécessaires.

3. Créez et initialisez d'abord votre modèle de réseau neuronal en tant que modèle séquentiel.

4. Ensuite, créez les couches du réseau…

5. Ajoutez la première couche LSTM avec un nombre fixe d'unités.
   - Définissez return_sequences=True afin de retourner la séquence complète dans la sortie.
   - Techniquement, return_sequences permet également de retourner la séquence cachée.
   - Par défaut, return_sequences=False dans les couches RNN de Keras<

6. Ajoutez une couche Dropout pour prévenir l'overfitting.
   - L'overfitting signifie que le modèle est limité au dataset utilisé et peut mal se généraliser à d'autres entrées.
   - La couche Dropout ignore aléatoirement certaines unités (neurones) lors de l'entraînement, en mettant essentiellement à 0 certaines connexions sortantes des unités cachées.
   - Ces unités ne sont pas complètement ignorées, mais chaque valeur a une probabilité d'être supprimée.

7. Ajoutez une deuxième, troisième, quatrième couche LSTM et Dropout, etc., selon vos besoins.

8. La dernière couche à ajouter est la couche Dense.
    - Cette couche connecte complètement les neurones de sortie aux neurones de la couche précédente.

9. Compilez le modèle.
    - Utilisez adam, qui est un optimiseur recommandé pour l'entraînement de modèles deep learning.

10. Utilisez la fonction de perte calculant l'erreur quadratique moyenne (MSE) pour comparer les valeurs observées aux valeurs prédites.

11. Entraînez le modèle sur vos données.
    Itérez sur plusieurs époques et définissez une taille de batch appropriée.

12. Préparez votre jeu de test.
13. Testez le modèle avec des prédictions.

14. Évaluez la performance du modèle.

15. Visualisez les données pour comparer les valeurs prédites avec les valeurs réelles à l'aide d'un graphique.

Ici, la couche Embedding mappe automatiquement les mots en vecteurs denses adaptés au LSTM.
